

Nome: Roberta Silva Dos Santos

Relatório de Desenvolvimento do Jogo RPG em Java

Introdução

Este relatório descreve o desenvolvimento de um jogo de RPG em Java, onde o jogador controla um personagem em um tabuleiro, enfrenta monstros, coleta itens e explora o mapa. O objetivo principal é derrotar o chefe localizado na última posição do tabuleiro. O jogo foi implementado utilizando conceitos avançados de Programação Orientada a Objetos (POO), como herança, polimorfismo, e classes abstratas, com uma interface gráfica criada com a biblioteca Swing.

Descrição do Diagrama de Classes

1. Entity abstract

Atributos

- int: x, y, health, damage, speed.

Métodos:

- Entity(int x, int y, int health, int damage, int speed)
- Entity()
- void setXY(dx: int, dy: int)
- int getX()
- int getY()
- int getDamage()
- int getHealth()
- int getSpeed()
- void setDamage(int damage)
- void setSpeed(int speed)
- void setHealth(int health)

2. Player extends Entity

Atributos:

- String name, type, up, down, left, right, baseImagePath, fightImage, bossFight
- Inventory inventory
- Int max, especialUso

Métodos:

- Player(int health, int damage, int speed, String type, String name, String baseImagePath)
- Player()
- void setMoviment(String type)
- String getBossFight()
- String getFightImage()
- String getBaseImagePath()
- String getName()
- String getType()

- String getDown()
- String getLeft()
- String getRight()
- String getUp()
- Inventory getInventory()
- int getMax()
- int getEspecialUso()
- void setEspecialUso(especialUso: int)

3. **Monster extends Entity**

Atributos:

- int level

Métodos:

- Monster(int x, int y, int damage, int speed, int level, int health)
- int getLevel()

4. **Tile**

Atributos:

- Monster monster
- Item item
- boolean visited
-

Métodos:

- Tile()
- void setMonster(Monster monster)
- Monster getMonster()
- void setItem(Item item)
- Item getItem()
- void markVisited()
- boolean isVisited()

5. **Board extends JPanel**

Atributos:

- int x, y playerX, playerY, i
- Tile[][] tiles
- boolean[][] visited

Métodos:

- Board()
- void draw(Graphics g)
- void setI(int i)
- int getI()

- void setPlayerPosition(int x, int y)
- void setVisited(int x, int y)
- int getX()
- int getY()
- Tile getTile(x: int, y: int)
- void paintComponent(Graphics g)

6. GamePanel extends JPanel

Atributos:

- Player player;
- Board board;
- JTextArea fightMessages;
- JScrollPane scrollPane;
- JPanel panel;
- JPanel sidePanel;
- JPanel centerPanel;
- JPanel imagePanel;
- JPanel infoPanel;
- JPanel controlPanel;
- JFrame frame;
- JLabel labelIcon;
- JLabel healthLabel;
- JLabel maxDamageLabel;
- JLabel speedLabel;

Métodos:

- GamePanel()
- void generateMonsters(int count)
- void placeMonstersOnBoard(Monster monster)
- void gerateItems(int count)
- void placeItemsOnBoard(Item item)
- void iniciarMenu()
- void showTutorial()
- void NovoJogo(JPanel panel, JFrame frame)

- void exibirClasse(String classe, String iconPath, int vida, int dano, int velocidade, String type, JPanel panel, JFrame frame)
- void jogo()
- void moviment(int dx, int dy, int movement)
- boolean fight(Monster monster)
- void adicionarLabel(String texto, JPanel painel)
- void limparFrame(JPanel panel)
- void abrirItemsFrame()
- void usarItem(String nomeDoItem)
- void abrirConfiguracoesFrame()

7. Inventory

Atributos:

- List<Item> items

Métodos:

- Inventory()
- void addItem(Item item)
- void removeItem(String item)
- List<Item> getItems()

8. Item

Atributos:

- String ImagePath, name
- int level

Métodos:

- Item(int level)
- int getLevel()
- String getName()
- String getImagePath()

Aplicação de Conceitos de Orientação a Objetos

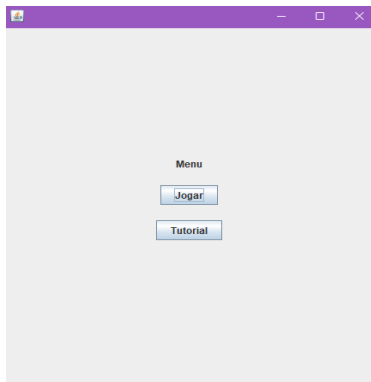
1. Herança: A classe Entity é uma classe abstrata que serve como base para Player e Monster, permitindo que ambas compartilhem atributos comuns como posição (x e y), vida (health), dano (damage) e velocidade (speed). Isso evita a duplicação de código e facilita a manutenção.
2. Polimorfismo: O conceito de polimorfismo é aplicado ao permitir que o método setXY da classe Entity seja utilizado tanto por objetos da classe Player quanto Monster. Assim, ambos

podem mudar sua posição no tabuleiro através do mesmo método, independentemente do tipo específico de objeto.

3. **Classe Abstrata:** A classe Entity é abstrata, o que significa que não pode ser instanciada diretamente. Isso garante que apenas classes derivadas, como Player e Monster, possam ser usadas para criar objetos. Este conceito foi crucial para definir comportamentos gerais em Entity, que são implementados ou estendidos nas subclasses.
4. **Interface Gráfica:** A interface gráfica foi implementada usando Swing. A classe Board é responsável por desenhar o tabuleiro, enquanto GamePanel gerencia a interação do jogador com o jogo. Este design modular permite a fácil adição de novos elementos gráficos e funcionalidades no futuro.

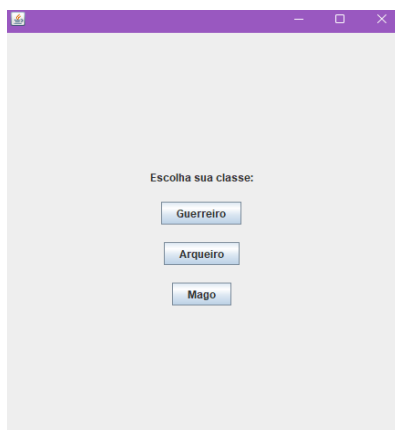
Passos

1. Iniciar o jogo:



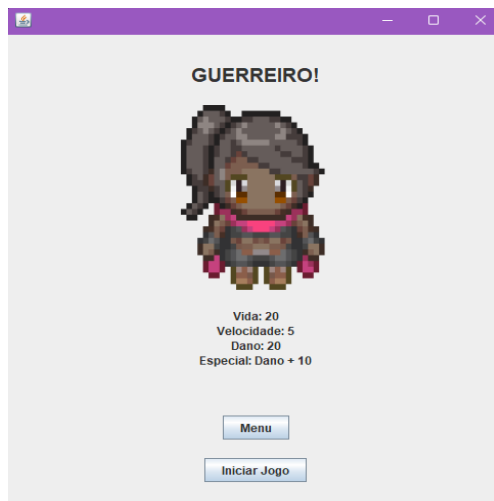
- O jogo começará com um menu inicial onde você pode escolher as opções de jogo, como iniciar uma nova partida ou acessar o tutorial.

2. Escolher o Personagem



- No menu, escolha o tipo de personagem que deseja jogar (Guerreiro, Arqueiro, Mago).

3. Confirmar escolha



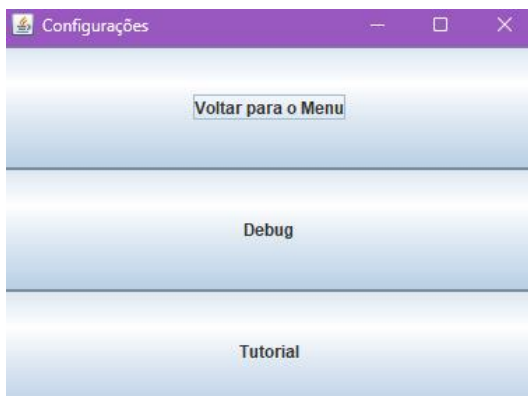
- Após selecionar o personagem, confirme a escolha para iniciar o jogo com o personagem escolhido ou volte para o menu para escolher novamente.

4. Movimentação



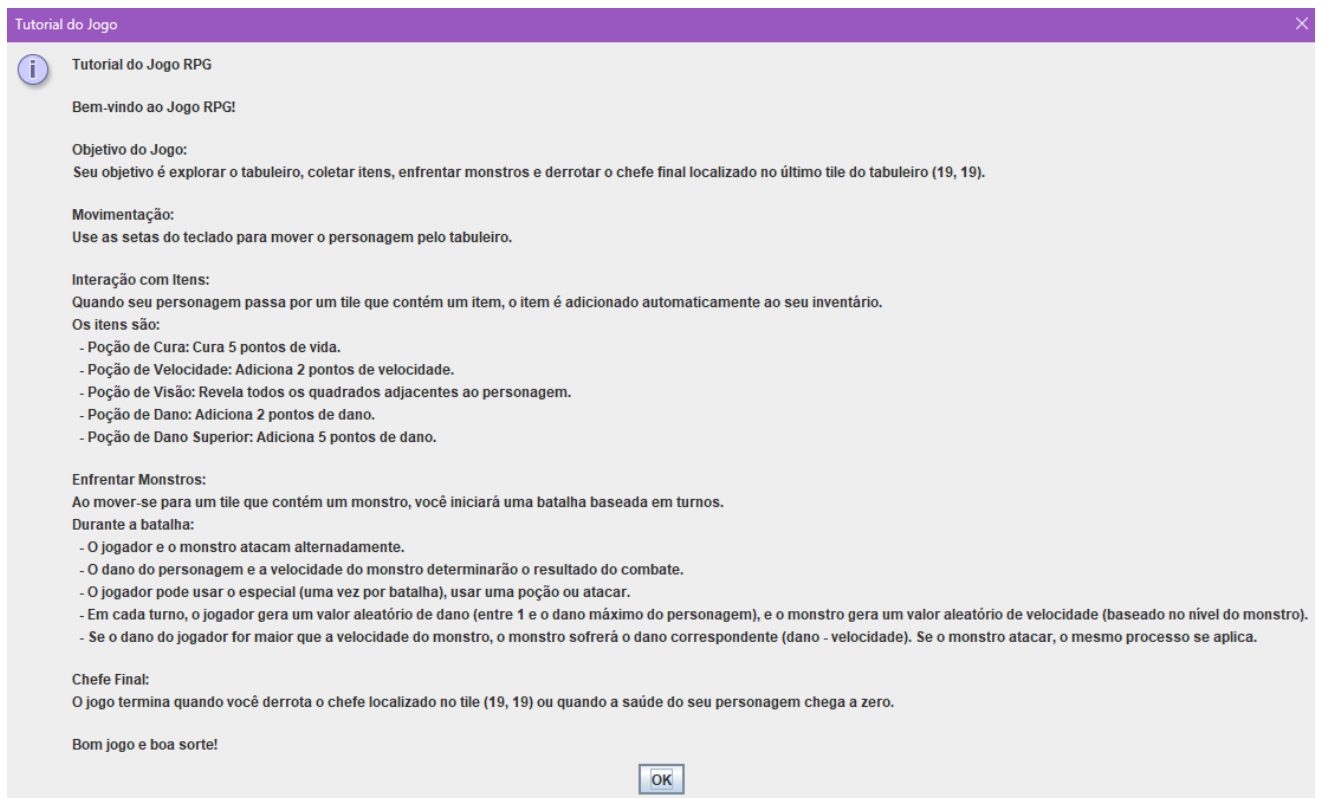
- Utilize os botões para mover o personagem pelo tabuleiro.
- O personagem se moverá em direções diferentes conforme a tecla pressionada.

5. Configurações



- volta ao menu inicial
- Debug para poder visualizar o mapa por completo
- Abrir tutorial

6. Tutorial



7. Visualização do mapa

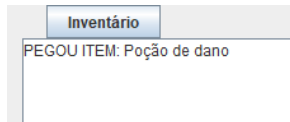
- No painel do jogo, observe o mini mapa localizado na parte esquerda. Ele mostrará a posição atual do personagem, o chefe e o estado dos tiles visitados. Caso esteja debugado (como na imagem) o mapa estará completamente visível.



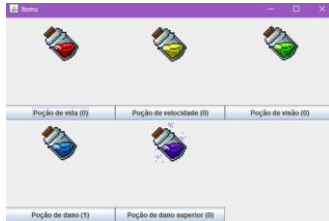
Legenda:

- Rosa: Jogador
- Laranja: monstros
- Verde: itens
- Azul: monstro e Item
- Cinza escuro: blocos vazios ou não visitados
- Cinza claro: lugares já visitados
- Vermelho: Chefe

8. Itens



- Ao passar sobre um item no tabuleiro, ele será automaticamente adicionado ao inventário do personagem.



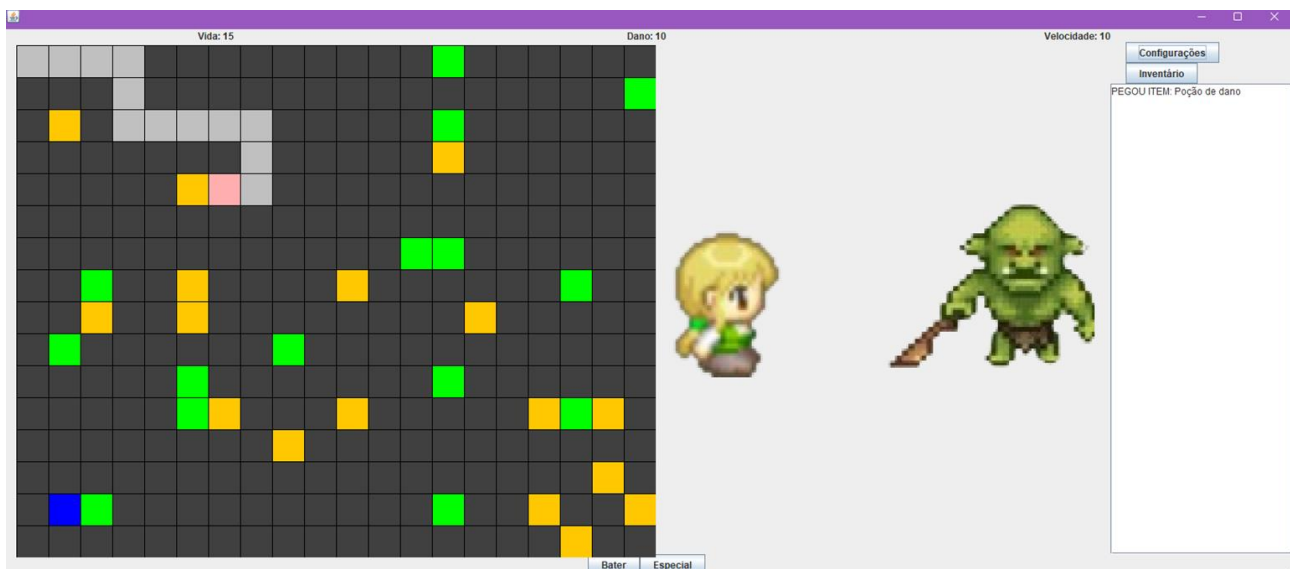
- Verifique com o botão de inventário

Cada poção tem um efeito de modo que:

- Cura: cura 5 pontos de vida;
- Velocidade: adiciona 2 pontos de velocidade;
- Visão: revela todos os quadrados adjacentes ao personagem;
- Dano: adiciona 2 pontos de dano;
- Dano superior: adiciona 5 pontos de dano;

9. Monstros

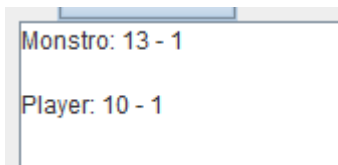
- Ao encontrar um monstro, a luta começará automaticamente.



- A luta é baseada em turnos, onde o personagem e o monstro atacam alternadamente.
- O jogador pode usar o especial uma vez que muda de acordo com a classe de personagem escolhida, usar alguma poção ou bater para usar seu turno.
- A vitória ou derrota em combate é determinada pela diferença entre o dano e a velocidade durante cada rodada. Quando o jogador ataca, um valor aleatório de dano é gerado, com um

máximo definido pelo dano do personagem, enquanto o monstro gera um valor aleatório de velocidade baseado no seu nível. Se o dano do jogador exceder a velocidade do monstro, a diferença é aplicada como dano ao monstro. O processo é invertido na rodada do monstro, onde seu dano é comparado com a velocidade do jogador, e a diferença é subtraída da saúde do personagem. A batalha prossegue até que a saúde de um dos combatentes chegue a zero, determinando o resultado do combate.

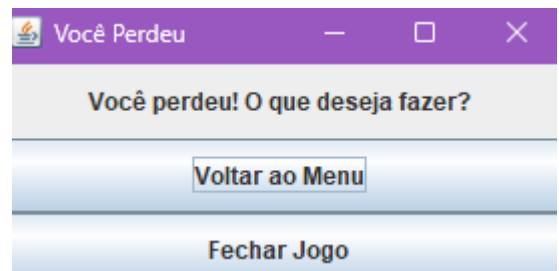
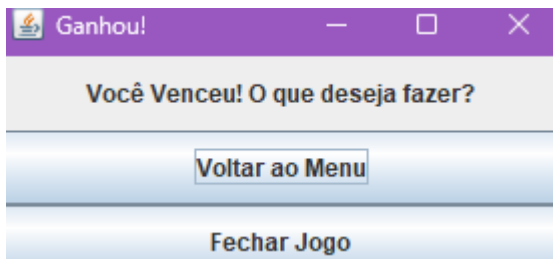
- Por exemplo: o jogador tira 10 de dano e o monstro tira 7 de velocidade, o monstro tomara um dano de 3 pontos de vida.



- A luta será atualizada por rodada de forma que, neste exemplo, o monstro tinha vida total de 13 e levou 1 de dano e o jogador tinha 10 de vida e tomou 1 de dano.

10. Finalização

- O jogo terminará quando o personagem tiver matado o chefe ou tiver sua saúde reduzida a zero.
- Você pode retornar ao menu principal para iniciar uma nova partida ou sair do jogo.



Relatório de Dificuldades Encontradas

Durante o desenvolvimento do jogo, enfrentei uma série de desafios que impactaram o progresso do projeto. A seguir, descrevo as principais dificuldades encontradas e as soluções implementadas.

1. **Uso de Sprites:** Inicialmente, tentei implementar o jogo utilizando sprites para criar animações e gráficos mais dinâmicos. No entanto, essa abordagem acabou se revelando bastante complexa e demandou um tempo considerável de desenvolvimento. A dificuldade em gerenciar e animar sprites resultou em um atraso significativo. Após muita tentativa e erro, decidi simplificar o processo optando por imagens estáticas, o que permitiu uma implementação mais rápida.
2. **Imagens de Fundo:** Outra dificuldade significativa foi a inclusão de imagens de fundo no jogo. O desafio estava em integrar corretamente essas imagens no painel do jogo, garantindo que elas fossem exibidas conforme o desejado. A complexidade aumentou devido às limitações e ao comportamento inesperado do Java ao manipular imagens de fundo. Apesar de várias tentativas e ajustes, a integração completa da imagem de fundo não foi alcançada como inicialmente planejado.
3. **Atualização dos Painéis:** A atualização dos painéis do jogo foi um outro obstáculo importante. A princípio, não compreendi totalmente como funcionava o mecanismo de atualização e redesenho dos componentes gráficos em Java. Após uma série de tentativas frustradas, descobri que a solução envolvia instanciar os componentes gráficos como atributos da classe `GamePanel`. Essa abordagem simplificou a modificação e a atualização dos painéis, permitindo que as mudanças fossem aplicadas de forma mais simplificada.
4. **Dificuldades Comuns:** Além dos desafios específicos mencionados, enfrentei dificuldades comuns como erros de lógica e a necessidade de refazer e testar várias vezes o código. Esses problemas são típicos no processo de desenvolvimento e fazem parte do ciclo de aprimoramento contínuo que qualquer programador enfrenta.

Conclusão

Apesar dos desafios enfrentados e do fato de que estou entregando este trabalho às vésperas do prazo final, posso afirmar com certeza que adorei o processo de desenvolvimento deste projeto. A experiência de criar um jogo RPG utilizando conceitos de Programação Orientada a Objetos foi muito boa, e era algo que já havia começado anterior a divulgação do trabalho, mas infelizmente tive que começar do zero. Cada etapa do desenvolvimento contribuiu significativamente para o meu aprendizado com programação orientada objeto.

Em relação a sugestões, eu não mudaria nada, já é a segunda vez fazendo a cadeira e desta vez me diverti mais fazendo este trabalho do que o último.